



电机功率模型与底盘功率控制



Team GMaster

作者

林文浩

邮箱

sgwlin4@liverpool.ac.uk

QQ

1225103523

目录

1	简介	3
2	概念阐述与符号定义	4
2.1	变量符号定义	4
2.2	Give current, Given current, 相电流 & 力矩	4
3	电机功率模型	5
3.1	电机机械功率	5
3.2	电机功率模型	6
3.3	模型拟合	7
4	功率控制	9
4.1	传统功率控制方式	9
4.2	功率再分配	9
5	部署	10
5.1	部署常见问题	10
6	未来优化的方向	10
7	总结	11
8	展望	11

1 简介

在超级电容出现后，传统的基于缓冲能量的功率控制已经不在适用。原因是一个理想的超级电容将会无条件的补偿超出最大功率的能量，在电容电量耗尽前，缓冲能量将不再能反映电机功率与最大功率的关系，因此此时无法对底盘功率做出任何限制。为了让操作手做出更多战术选择，实现主动释放超级电容能量，我们迫切需要一个新的底盘功率控制算法。

控制底盘功率的最直接的手段便是建立电机功率模型，直接计算出在当前时刻发送多大的力矩可以让电机消耗指定的功率。从而令电机消耗任意大小的功率。

本算法基于广东工业大学给出的电机功率公式并在此基础上加以改进，优化了电机模型公式的符号问题，使其考虑到了电机减速时可以输出的额外力矩。为了减少调参时间，使用 Matlab 对电机参数进行拟合。

同时发现现在网络上缺少关于 C620 电调解释资料缺乏，本文档还详细的解释了 Give current, Given current, 相电流 & 力矩的相关概念。

2 概念阐述与符号定义

2.1 变量符号定义

在本文之中，我们规定，顺时针方向的力矩为正，逆时针方向的力矩为负，顺时针方向的转速为正，逆时针方向的转速为负，功率为正表示消耗功率，功率为负表示输出功率。转速均为电机转子转速，来自 C620 电调反馈值。

2.2 Give current, Given current, 相电流 & 力矩

我们首先讨论，发送给电机的控制参数究竟代表什么。give_current 和 given_current 在官方代码代表着发送给电机的控制量和电机反馈的控制量。

我们使用的电调与电机套装为 C620 搭配 M3508 无刷电机，在对 M3508 进行控制时，我们是通过给 C620 电调发送力矩电流控制值来控制其力矩。根据手册所写，发送电流值的范围为-16384~0~16384，即 give_current 的范围，对应电调的输出力矩电流为-20A~0~20A。于是我们有

$$I_{\tau} = \frac{20}{16384} \times I_{cmd} \quad (1)$$

I_{τ} 为力矩电流，单位 A， I_{cmd} 为电调力矩电流控制值。力矩电流为电机用于输出力矩的电流，并非电机相电流，不能使用力矩电流乘电压得到电机的输入功率，力矩电流与力矩的关系如下

$$\tau = K_T \times I_{\tau} \quad (2)$$

K_T 为力矩电流常数，与电机的特征参数，我们可以在 M3508 电机使用手册中找到。但是手册中的转矩常数描述的为电机减速箱输出轴的力矩，由于电机反馈的转子转速，为了方便计算，我们这里需要通过减速比计算得到电机转子的转矩常数。

$$K_T = 0.3 \times \frac{187}{3591} = 0.01562 N \cdot m / A \quad (3)$$

综上，因为转矩和转矩电流控制值的完全可以相互转换，在得到目标转矩后可以简单求得转矩电流控制值。转矩电流控制值本质上就是在设置电机的目标转矩。为了方便表述减少歧义，下文建模时将不再提及转矩电流控制值。在最后求得转矩后进行转化即可。

3 电机功率模型

3.1 电机机械功率

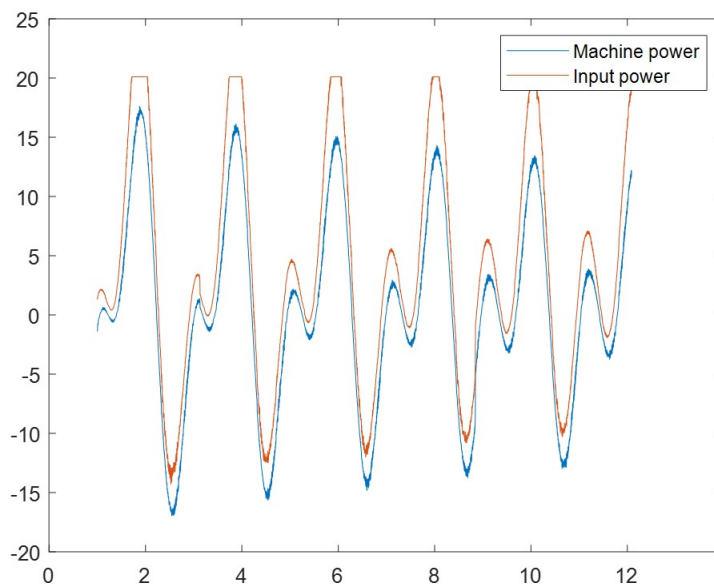


图 1: 电机机械功率与输入功率

注：输入功率出现失真是因为电流超过 INA226 最大采样电流

无刷电机的输入功率主要由机械功率，铜损，磁损耗，控制器静态功耗组成。其中机械功率占主要部分，其大小为：

$$P_m = \frac{\tau\omega}{9.55} \quad (4)$$

ω 为转子转速，单位 RPM，9.55 为转化系数，推导可自行搜索，在此不在赘述。

我们队伍之前的两版功率控制都是通过比较发送的四个电机总的力矩目标值与我们预先设置好的总力矩阈值进行比较，将各个电机的目标力矩缩放到阈值之下来对功率进行限制。

但是观察上式，电机的消耗功率不仅仅只和力矩有关，还和电机当前转速有关。这导致上文中的力矩阈值在低速情况下严重偏小，造成底盘起步疲软无力，加速缓慢。

同时，我们也不能只使用机械功率进行功率控制，观察上图可以发现，机械功率和总功率在高力矩或高转速情况下。若只使用机械功率进行控制，会造成严重的超功率问题。一个理想的功率控制应该可以使电机恒功率启动，即保证从加速到匀速整个过程电机均工作在最大功率处，以实现理论最大加速度。

3.2 电机功率模型

本文想要对于 M3508 建立一个功率模型，通过给与这个模型转矩，当前转速等信息，得到电机的输入功率。或者通过输入功率，当前转速，得到应该输出的力矩大小，进而得到力矩阈值。

我们在这里假设 C620 电调的力矩电流环是完美的，即它对于发送给他的任何力矩 C620 电调能够完美的瞬间相应。笔者通过采集比较电机反馈的真实力矩电流数据与发送数据也可以印证这一假设是成立的。

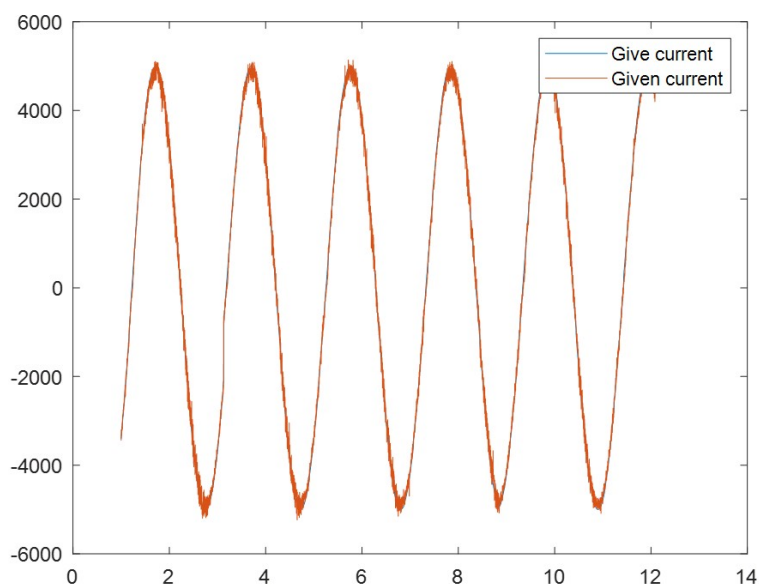


图 2: 力矩电流发送至与力矩电流反馈值

$$P_{in} = P_m + k_1\omega^2 + k_2\tau^2 + a \quad (5)$$

通过查阅广东工业大学给出的功率控制方案，电机的输入功率除机械功率之外，还与转速和力矩的二次项有关。他们的相关关系可以由上面这个经验公式给出。对于这个模型，笔者查看了很多资料并没有找到该式子的来源，推测为经验公式，但是该模型在 RM 队伍中广泛使用，应该是有一定原因的。

3.3 模型拟合

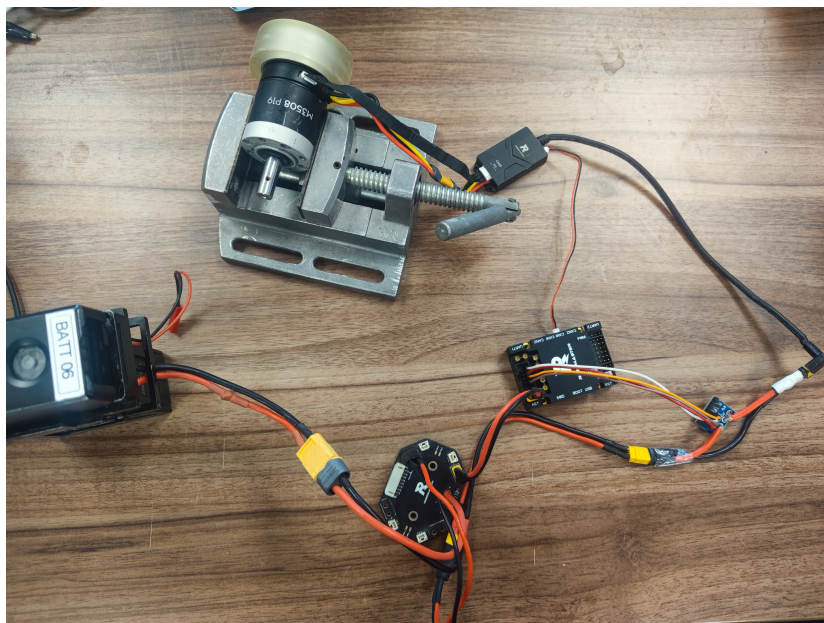


图 3: 数据采集硬件平台

笔者也选取该模型作为无刷电机功率模型。与广工工业大学不同的是，笔者对于电机的机械功率部分直接使用公式 4 计算，为了更为准确的得到 k_1, k_2, a 的大小，笔者使用 RM C 型开发板对 C620 电调发送不同频率幅值的正弦力矩电流控制值，并对电机施加随机大小的负载，使用 INA226 采样实时的电调消耗电流与总线电压。然后使用 STM32CubeMonitor 将真实电流值，力矩控制值，转子转速导出保存至 CSV 文件。接下来使用 Matlab 的 fit 和

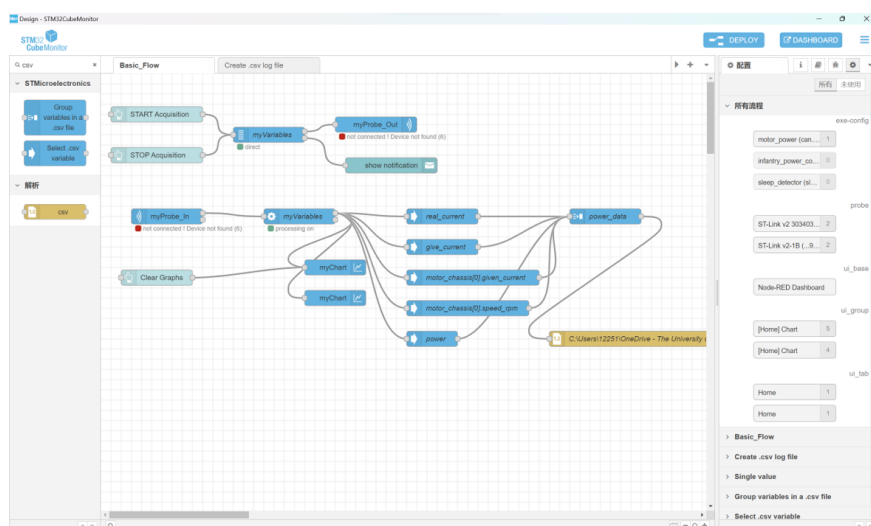


图 4: STM32CubeMonitor 软件视图

fittype 函数对其进行进行拟合，并通过不同组数据交叉验证数据仿真效果，最终选取了一个仿真程度较好的参数。

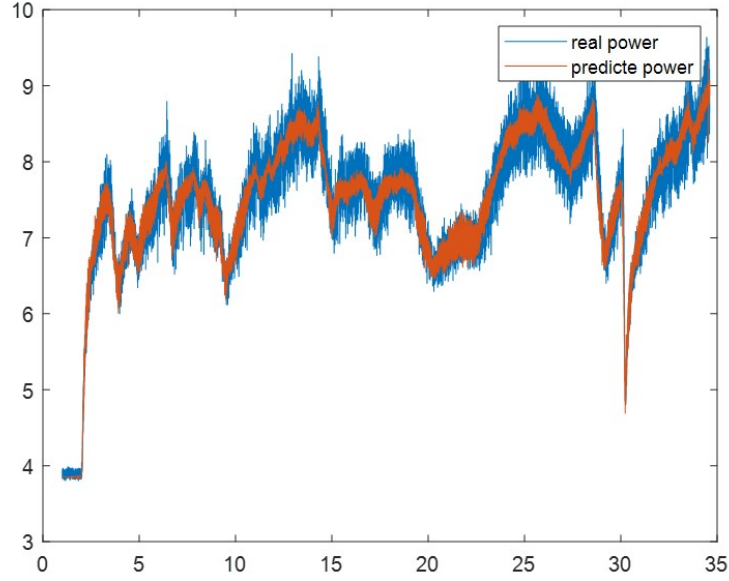


图 5: 预测输出功率与实际输出功率

在得到 k_1 , k_2 , a 的参数后, 我们可以对该模型输入当前转速, 原始力矩电力控制值, 便可得到不加限制的原始功率, 即若直接对电调发送该值, 电机在此时刻的功率。我们接下来需要做的是, 若想要电机工作在功率 P_{max} , 应该发送多大的目标力矩。将 P_{max} 代入模型, 整理

$$k_1\tau^2 + \frac{\omega\tau}{9.55} + k_2\omega^2 + a - P_{max} = 0 \quad (6)$$

观察该式为一个关于转矩的二次方程, 其他量均为已知量, 对转矩进行求解, 我们可以得到

$$\tau = \frac{-\frac{\omega}{9.55} \pm \sqrt{(\frac{\omega}{9.55})^2 - 4k_1(k_2\omega^2 + a - P_{max})}}{2k_1} \quad (7)$$

在这里会遇到二次方程两个解的问题, 在起初笔者直接使用正号的解作为目标力矩的大小, 然后依据原始力矩的正负符号对其重新赋值, 在实际效果表现很差。后来意识到是这里出现的问题。因为我们在上文已经严格规定了符号的正方向并全程使用符号计算, 未使用绝对值, 因此这里的两个解一定都是有意义的, 不能先入为主的舍掉。

然后笔者重新分析了该模型, 意识到, 在同一时刻, 确实存在一正一负两个力矩值, 它们令该电机消耗的功率都为 P_{max} ! 而这二次方程的两个解就刚好对应着一正一负的两个目标力矩值。

同时若我们对这个方程进行简单的定性分析, 在某一时刻若转速为正, 若让它消耗同一功率, 其输出的正的加速力矩绝对值要小于负的减速力矩的绝对值。也就是说, 在减速时, 相同功率, 我们可以输出更高的减速的力矩, 这和电动机反电动势的原理是一致的。

在解决这个问题后, 我们就可以依据原始 PID 出力矩的方向选择对应的解来进行计算。防止出力矩方向相反。

4 功率控制

4.1 传统功率控制方式

在得到该模型后，最简单的方案是可以将功率上限除以四，对每个电机均匀的分配最大功率的四分之一，但是这会导致一个问题，由于不同电机负载不同，转速不同，但对其分配相同的功率会造成功率浪费，或者在加速时无法由于负载不同但是功率是均分的，造成底盘无法走直线，因此该方案首先被排除。

第二种方案是广东工业大学采用的，设一个总力矩缩放系数 k ，将模型公式参数全部变成绝对值，然后计算缩放系数，详细过程如下，其中 τ_{cmd} 是原始的发送力矩。

我们根据(2)式来实现功率限制。已知：最大功率 P_{max} 、各电机当前转速 ω_{real} 和电机 PID 计算出来的原始力矩指令 τ_{cmd} 。当 τ_{cmd} 将使 P_m 高于 P_{max} 时，设有一缩放系数 k ，令 $\tau'_{cmd} = k\tau_{cmd}$ ，使得 τ'_{cmd} 满足：

$$P_{max} = \sum |\omega_{real}\tau_{cmd}| + k_1 \sum \tau_{cmd}^2 + k_2 \sum \omega_{real}^2$$

则可由上式计算出 k 的值：

$$k = \frac{-\sum |\omega_{real}\tau_{cmd}| + \sqrt{\sum (\omega_{real}\tau_{cmd})^2 - 4k_1(\sum \tau_{cmd}^2)(k_2 \sum \omega_{real}^2 - P_{max})}}{2k_1 \sum \tau_{cmd}^2}$$

最终给电机的力矩指令即为 $\tau'_{cmd} = k\tau_{cmd}$ 。

k_1 、 k_2 的调试方法：实时读取裁判系统反馈的底盘功率。先让底盘电机堵转，调整 k_1 使底盘实际功率大致与限制功率相等。再让机器人原地小陀螺，调整 k_2 使底盘实际功率大致与限制功率相等。

图 6: 广东工业大学在线文档

这种方法的问题在于由于我们在推导模型之前对符号严格定义过，该模型本身是能够计算出某些情况下电机减速产生反电动势所输出的负功。但是全部使用绝对值进行计算，意味着无法考虑这部分负功且会使理论功率和消耗功率偏差极大，因为在负功情况下根本不消耗功率，但是使用该式子计算依然会得到一个较大的“正”功率，这会导致对于功率利用的严重不充分。

4.2 功率再分配

本功率控制算法创新性的提出了第三种功率控制算法，使用功率再分配，对电机功率进行缩放。我们首先使用该模型正向计算出各个电机的功率，得到功率缩放系数 k 。若 $k > 1$ 则不缩放。

$$k = \frac{P_{max}}{\sum P_{cmd}} \quad (8)$$

令 $P' = k \times P_{cmd}$ ，得到重新分配过的电机功率，然后将该功率依据原始力矩方向代入到前文公式中的两个解之一，得到缩放后的力矩大小。在这里我们对输出负功的电机不加以缩放，由于它本身已经不消耗功率了。该方法的好处是不光可以减少输出负功的电机功率带来的误差，还可以相应的使总输出功率略微变大，尝试令其他电机吸收这部分负功，因为在计算原始功率时，输出负功是以一个负项加到总功率当中的。

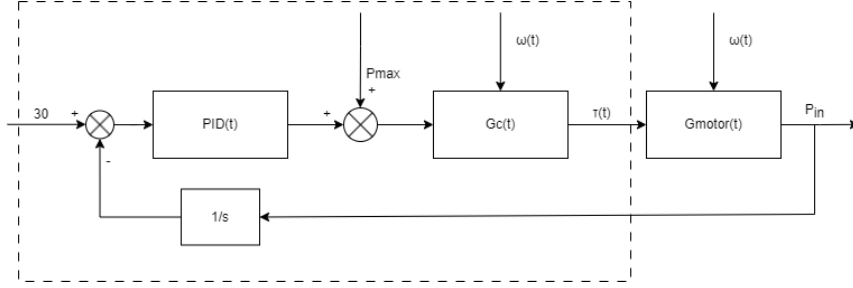


图 7: 控制框图

上图为总功率控制框图，虚线内为软件部分，其中包含一个缓冲能量闭环，Gmotor(t) 为电机真实的物理模型。

5 部署

在使用 Matlab 拟合和部署在车上时，为了减少数据处理的，直接使用的力矩电流发送值 I_{cmd} 进行的拟合与计算。为了降低大家阅读代码的难度，我在下方对其进行了推导。

$$\begin{aligned}
 P_{in} &= \frac{K_T I_\tau \omega}{9.55} + k_1 \omega^2 + k_2 I_{cmd}^2 \\
 &= \frac{K_T}{9.55} \times \frac{20}{16384} I_{cmd} \omega + k_1 \omega^2 + k_2 I_{cmd}^2 \\
 &= C_T I_{cmd} \omega + k_1 \omega^2 + k_2 I_{cmd}^2
 \end{aligned} \tag{9}$$

参数 C_T 笔者命名为力矩电流系数，用于在 C620 搭配 M3508 动力套件下，从力矩电流乘转速 (RPM) 转化为功率，大小为 1.996×10^{-6} 。

5.1 部署常见问题

1. 拟合数据时电机不要空载，电机空载时力矩电流控制值不等于力矩电流。
2. 走不直把功率均分看看

6 未来优化的方向

笔者起初认为，即使不同新旧电机的阻尼阻值不同，但是它们都应该为一个定值，所有 M3508 搭配 C620 的电机功率模型都是相同的，即这个模型认为被用于获取数据的电机，和所有部署在机器人底盘上的电机功率性能全部一致。

在写该文档的时候我了解到，这个观点是错误的，因为不同电机的有不同齿轮箱阻尼系数，它会产生一个随转速二次相关的力矩，反映到该模型中就是不同电机的 k_1 参数不同。

笔者目前还没有学过电机学，受限于笔者的知识储备，目前无法对该电机模型进行优化。在使用时该算法只能保证用于获取数据的电机性能和实际步兵性能一致，或者直接从步兵上获取拟合参数，并保证四轮电机保养状态一致。在未来，需要对该电机功率模型的经验公式进行进一步优化，探讨有无其他相关项。

同时该功率控制算法由于对功率限制过于严格，这会与平衡步兵的平衡控制算法冲突，导致平衡步兵在高移速的时候无法再输出足够的功率改变倾角进行减速，会使平衡步兵出现滑铲的现象。为了兼容平衡步兵，这需要额外的算法对输出功率进行调度，在减速时及时调高功率上限，让超级电容介入以正常减速。

7 总结

本文在广东工业大学原有模型基础上进行了分析与推导，建立出了更加完善的电机功率模型。该模型可以更好的利用电机减速时更好的利用电机所产生的反电动势，同时使用 Matlab 对参数进行拟合，减少了调参时间。

受限于备赛时间紧迫，机器人的远程参数仅通过远程仿真器查看，未能保存，因此该未能在这里展示实际的功率控制效果。该模型最终可以实现在任何工况下，使缓冲能量稳定在 30，并且电容容值为满。这意味着底盘消耗的功率永远在所设置的最大功率上下浮动。在实际比赛中，可以将该功率设置为略微大于底盘最大输入功率，以一个稍小的功率使用电容的能量，防止底盘静止时，电容容量为满无法对电容充电的情况，以提高整体能量利用率。

8 展望

笔者从大一开始在 A 型开发板上写功率控制，写了一个很粗糙的力矩电流缩放，比完赛了才发现官方在 C 板上有一个很不错的功率控制，使用的是缓冲能量加力矩电流缩放。大二的时候开始写可以应用到超级电容系统中的功率控制，写了一个很不好用的控制算法，也是基于力矩电流缩放的，有着很严重的起步昏厥的问题。直到分区赛结束，我才了解到广东工业大学有了一套更优雅的功率控制算法。到了今年分区赛，我们队伍还是使用着我的那套很不好用的功率控制，在比赛场上有着严重的起步昏厥的问题。

于是我暑假回国之后便自告奋勇承担起了新功率控制算法的研发任务。整套工作历时两周完成。在我写功率控制的这三年里，我无数次的想要在论坛上搜索其他队伍是如何做功率控制的，什么是 give current 什么是 given current 他们的关系是什么，但是都只找到 19 年 RM 圆桌的相关内容，而且其中的内容十分老旧且不严谨。

这也是我为什么决定将该算法开源，写文档，出视频讲解的内容。这个算法可能其他强队很早就已经在用的，甚至在强队看来不一定是最先进的，但是我应该是第一个将它专项完成开源出来，同时细致的写文档的。因为我深知功率控制是萌新队伍最难解决的问题。个人认为 RoboMaster 作为一个机器人比赛对于电控的要求不太高，或者说电控和硬件很难

为比赛获得决定性优势，除了平衡步兵之外，这几年几乎是得算法得自瞄者得天下。论坛上关于自瞄的开源比比皆是，但却再少有关于一些低级技术问题的专项开源讲解了。

我们队伍从 21 年邀请赛一步步走来，深感萌新队伍的起步困难，学习资料少，闭门造车不出成果，论坛分析内容过于老旧，其他队伍的整车电控开源内容庞杂，缺少文档，不好查找，难以找到需要的具体技术。我想在此呼吁各个队伍针对于一些技术细节，比如功率控制，动态 UI 绘制，裁判系统通信，舵轮控制甚至是小陀螺进行有针对性的开源和讲解，这些才是真正的对萌新队伍有利的开源。